Predmet: "Vještačka inteligencija"
Laboratorijska vježba 11: Optimizacija roja čestica
Odgovorna nastavnica: Vanr. prof. dr Amila Akagić



Sadržaj vježbe:

1 Cilj vježbe **1**

2 Uvod **1**
2.1 PySwarms biblioteka 1
2.2 Osnovna optimizacija korištenjem PySwarms biblioteke 2
2.3 Optimizacija vlastite objektivne funkcije 2

3 Zadaci za rad u laboratoriji **5**

1 Cilj vježbe

Cilj ove laboratorijske vježbe je upoznavanje s osnovnim konceptima inteligencije roja, kroz implementaciju PSO algoritma korištenjem biblioteke PySwarms, koristeći ga kao optimizator za jednostavnu neuronsku mrežu koja rješava problem klasifikacije.

2 Uvod

Optimizacija roja čestica (eng. *particle swarm optimization* - PSO) je metaheuristički algoritam koji počiva na jednostavnom principu primijećenom u životinjskom svijetu. Naime, u situacijama kad skupina životinja pokušava postići neki cilj, svaka će individua nastojati da radi prema ostvarenju tog cilja u skladu sa svojim vlastitim iskustvom i instinktima, ali će posmatrati i stanje ostalih individua, prilagođavajući se ukoliko primijeti da druge jedinke na neki način ostvaruju bolji rezultat. Taj koncept je ovdje u potpunosti prenesen na jedan od osnovnih problema u vještačkoj inteligenciji: problem pronalaska optimalne regije nekog višedimenzionalnog prostora pretraživanja.

PSO počinje od seta potencijalnih rješenja, koja nazivamo česticama. Svaka čestica u nizu iteracija mijenja svoj položaj u višedimenzionalnom prostoru od interesa na osnovu dvije važne informacije. Prva je najbolji rezultat koji je ta konkretna čestica ostvarila (eng. *personal best*), a druga je najbolji rezultat koji je ostvarila najuspješnija susjedna čestica (eng. *global best* - iako ovaj pojam implicira da u susjedne čestice ubrajamo čitav roj, što ne mora biti način implementacije algoritma, pojam se historijski održao). U slučaju da algoritam konvergira, čestice će se naći okupljene oko optimalnog položaja nakon nekog broja iteracija.

Kao i genetičke algoritme, PSO je moguće primijeniti na jako široku klasu problema, kao što su klasifikacija, prepoznavanje šablona, planiranje, modeliranje bioloških sistema, obrada signala i sl.

2.1 PySwarms biblioteka

PySwarms predstavlja toolkit namijenjen za istraživanje i edukaciju čija je funkcija kreiranje PSO optimizatora na jednostavan način, uz dobro definisane konvencije i konzistentnost API-ja. Instalacija se može obaviti standardno kroz pip:

```
1 pip install pyswarms
```

Prilikom importovanja, zahtjev je da se prethodno importuje i NumPy.

2.2 Osnovna optimizacija korištenjem PySwarms biblioteke

Primjer koda izložen u nastavku optimizira Rastriginovu funkciju. Iskoristićemo ga da se upoznamo sa sintaksom i procedurom korištenja PySwarms.

```
import numpy as np

import pyswarms as ps
from pyswarms.utils.functions import single_obj as fx

max_bound = [5.12, 5.12]
min_bound = [-5.12, -5.12]

bounds = (min_bound, max_bound)

options = {'c1': 0.5, 'c2': 0.3, 'w': 0.9}

optimizer = ps.single.GlobalBestPSO(n_particles=10, dimensions=2, bounds=bounds,
    options=options)

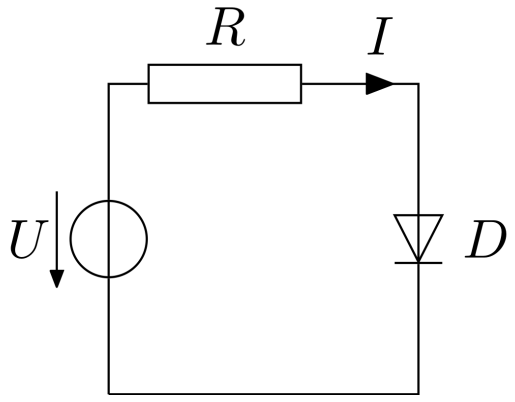
optimizer.optimize(fx.rastrigin, iters=1000)
```

Nakon importovanja NumPy i PySwarms biblioteka, importuje se i `single_obj` modul, unutar koga se nalaze funkcije koje mogu služiti za testiranje optimizacionih algoritama. Jedna takva funkcija je i Rastriginova, s kojom smo se upoznali na laboratorijskoj vježbi 9, gdje je dat i njen analitički zapis. Pošto je funkcija ograničena na interval $[-5.12, 5.12]$ u obje dimenzije, kako bismo čestice svog roja održali unutar prostora na kome je funkcija definisana, možemo to uraditi pomoću `bounds` parametra, definisanog kao u kodu iznad. Pored `bounds`, u kodu je dat i `options` *dictionary*, međutim, za potrebe svega što ćemo raditi na vježbi, on može ostati nepromijenjen. Za model je važno odabrati i broj čestica u roju, definisan sa `n_particles`, te naznačiti broj nezavisnih promjenjivih funkcije koja se optimizira kroz parametar `dimensions`.

Ovako određeni hiperparametri se prosljeđuju kreiranoj instanci PSO optimizatora, nakon čega slijedi poziv njegove optimizacijske metode, kojoj se kao parametri prosljeđuju objektivna funkcija (tj. funkcija koju optimiziramo), te broj iteracija (ažuriranja položaja svake čestice u roju). Metoda vraća rješenje u vidu položaja čestice koja je postigla najbolji rezultat i njoj odgovarajuće vrijednosti objektivne funkcije. Ovi koraci predstavljaju osnovu korištenja PySwarms biblioteke za proces optimizacije.

2.3 Optimizacija vlastite objektivne funkcije

Naravno, objektivne funkcije možemo definisati i sami, te ih optimizirati na ovaj način. U narednom primjeru ćemo pokušati pronaći električnu struju koja teče kroz kolo na slici 1, tako što ćemo rješavanje nelinearne jednačine koja ga opisuje modelirati kao optimizacioni problem.



Slika 1: Električno kolo sačinjeno od diode, otpornika i naponskog izvora.

Po Kirchhoffovom zakonu za napone, znamo da je napon naponskog izvora U (poznata vrijednost, $10V$) jednak sumi

napona na otporniku V_R (poznatog otpora, 100Ω) i napona na diodi V_d.

$$U = V_d + V_R$$

Da bismo odredili struju I koja teče kroz kolo, potrebno je napone V_R i V_d izraziti u funkciji struje. Poznato je da je u skladu s Ohmovim zakonom napon V_R moguće zapisati kao:

$$V_R = RI$$

S druge strane, dioda je izrazito nelinearan element čiji se rad može opisati kroz relaciju:

$$V_d = V_T \ln \left| \frac{I}{I_s} \right|$$

Vrijednosti I_s i V_T su konstante čije su vrijednosti $9.4pA = 9.4 \cdot 10^{-12}A$ i $25.85mV = 25.85 \cdot 10^{-3}V$, respektivno. Prema tome, relacija kojom je cijelo kolo sada opisano ima oblik:

$$U = V_T \ln \left| \frac{I}{I_s} \right| + RI$$

Da bismo ovu nelinearnu jednačinu preveli u oblik pogodan za optimizaciju, prebacićemo sve članove jednačine na istu stranu. Dobijeni izraz će predstavljati objektivnu funkciju za naš problem: biće nam cilj da pronađemo vrijednost struje za koju je vrijednost objektivne funkcije jednaka nuli. Ukoliko uzmemo apsolutnu vrijednost od tako definisane funkcije, posmatrani problem efektivno postaje problem minimizacije funkcije, budući da minimum funkcije postaju one tačke u kojima je ona jednaka nuli. Konačno, objektivna funkcija se može opisati relacijom:

$$c = \left| U - V_T \ln \left| \frac{I}{I_s} \right| - RI \right|$$

Ova funkcija se sada može u Pythonu prikazati na sljedeći način:

```
def cost_function(I):

    U = 10
    R = 100
    I_s = 9.4e-12
    v_t = 25.85e-3

    c = abs(U - v_t * np.log(abs(I[:, 0] / I_s)) - R * I[:, 0])

    return c
```

Pretpostavljajući da su prethodno odgovarajući moduli importovani, kod kojim obavljamo optimizaciju je analogan onom koji smo u prethodnim primjerima koristili.

```
options = {'c1': 0.5, 'c2': 0.3, 'w':0.9}

# Call instance of PSO
optimizer = ps.single.GlobalBestPSO(n_particles=10, dimensions=1, options=options)

# Perform optimization
cost, pos = optimizer.optimize(cost_function, iters=200)
```

Postavke su odabrane na način da će u prosjeku greška određivanja struje I biti reda veličine 10^{-5}. Da bismo okvirno provjerili ovaj rezultat, možemo funkciju `cost_function` plotati unutar intervala $[0, 0.1]$, međutim, preciznija provjera se može odraditi korištenjem nelinearnih solvera. U tu svrhu možemo iskoristiti `fsolve` iz biblioteke `scipy`.

```
from scipy.optimize import fsolve

def cost(I):
  return abs(10 - 25.85e-3 * np.log(abs(I / 9.4e-12)) - 100 * I)

initial_guess = 0.09
```

```
7
8  I = fsolve(func=cost, x0=initial_guess)
9
10 print(I[0])
```

Pri provjeri rješenja za struju I koja smo dobili za ova dva različita načina rješavanja, možemo utvrditi veliku tačnost oba, ali i činjenicu da je za rad solvera potrebno neko inicijalno rješenje kao ulazni podatak, što s PSO algoritmom nije slučaj. Također, korištenje ovog solvera zahtijeva da funkcija koja se optimizira prima strogo jednu vrijednost, za razliku od prethodno definisane cost funkcije za PSO, koja je mogla da primi Python niz ili tuple.

3 Zadaci za rad u laboratoriji

Dosad izložene specifičnosti korištenja PySwarms biblioteke su sasvim dovoljne za izradu zadatka u nastavku.

Zadatak 1 - Optimizacija neuronske mreže

U ovom primjeru ćemo treniranje neuronske mreže izvršiti korištenjem PSO algoritma.

Mreža će koristiti Iris Dataset iz `scikit-learn` modula. Importovaćemo ga, te odmah odvojiti značajke i labele. Također, kako su labele irisa u `data.target` date kao vektor dimenzija 150×1 u koji su smještene vrijednosti 0, 1 i 2, u zavisnosti od vrste irisa, podaci su one-hot enkodirani na osnovu te strukture. Osim toga, odvojili smo dostupne podatke na trening i testni skup, pomoću poznate funkcije `train_test_split`.

```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target

y_new = np.zeros([X.shape[0], 3])

for i in range(X.shape[0]):
  y_new[i, y[i]] = 1.

y = y_new

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)
```

Prisjetimo se da prilikom treniranja neuronske mreže zapravo optimiziramo vrijednosti težina i otklona za sve neurone u mreži, te da se vrijednosti dobijene treniranjem kasnije evaluiraju s obzirom na odabranu funkciju gubitka. Prema tome, PSO algoritam će raditi istu stvar: svaka čestica u roju će predstavljati jedno potencijalno rješenje, odnosno, jedan set matrica težina i vektora otklona.

Kako bismo to omogućili, kreiraćemo tri funkcije: prva će definisati mrežu i njenu strukturu, druga će pozivom prve omogućiti prolazak kroz mrežu, a zatim računati vrijednost funkcije gubitka za jednu česticu, dok će treća funkcija pozivajući drugu isti proces provoditi za kompletan roj. Prva funkcija je data u nastavku.

```
n_inputs = 4
n_hidden = 20
n_classes = 3

num_samples = 150

def probs_function(p, X):
    W1 = p[0:80].reshape((n_inputs,n_hidden))
    b1 = p[80:100].reshape((n_hidden,))
    W2 = p[100:160].reshape((n_hidden,n_classes))
    b2 = p[160:163].reshape((n_classes,))

    z1 = X.dot(W1) + b1
    a1 = np.tanh(z1)
    logits = a1.dot(W2) + b2

    exp_scores = np.exp(logits)
    probs = exp_scores / np.sum(exp_scores, axis=1, keepdims=True)

```

```
    return probs
```

Sama mreža koju ćemo konstruisati je dosta jednostavna: imaće ulazni sloj sa 4 ulaza (što pokriva sve značajke ovog dataseta), zatim jedan skriveni sloj sa 20 neurona, čija je aktivacijska funkcija $\tanh(x)$, te izlazni sloj od 3 neurona, sa *softmax* aktivacijskom funkcijom, koju koristimo za višeklasnu optimizaciju.

Posmatrajmo dimenzije mreže. Matrica težina između ulaznog i skrivenog sloja ima $4 \cdot 20 = 80$ parametara, dok matrica težina koja spaja skriveni i izlazni sloj ima $20 \cdot 3 = 60$ parametara. Vektori otklona imaju 20 i 3 parametra, respektivno, tako da je ukupan broj parametara za cijelu mrežu 163. Prema tome, svaka čestica predstavlja niz od 163 realna broja. U kodu iznad, parametar funkcije p predstavlja česticu iz koje se dobijaju parametri $W1$, $b1$, $W2$ i $b2$, koji predstavljaju matrice i otklone prvog i drugog sloja, respektivno.

Prolazak kroz mrežu odgovara skalarnom množenju matrica sloja i ulaza u sloj, zatim sabiranju otklona s dobijenim proizvodom, nakon čega se na rezultat primjenjuje aktivacijska funkcija sloja. Izlaz iz ove funkcije je matrica koja ima onoliko redova koliko ih je imao skup značajki proslijeđen kao parametar funkcije X (tj. trening ili testni skup), te 3 kolone: za svaki iris u proslijeđenom skupu, daju se vjerovatnoće pripadnosti svakoj od 3 moguće skupine.

Forward-propagaciju, te računanje vrijednosti funkcije gubitka za jednu česticu, provodimo kroz funkciju datu ispod.

```
from keras import losses

def forward_prop(params):

    probs = probs_function(params, X_train)

    cce = losses.CategoricalCrossentropy()
    loss = cce(y_train, probs).numpy()

    return loss
```

Primijetimo da je funkcija gubitka, kao i inače za probleme višeklasne klasifikacije, kategorička unakrsna entropija. Konačno, posljednja funkcija, ujedno i objektivna za PSO optimizator, će za sve čestice u roju pozivati `forward_prop` funkciju. Izlaz iz ove funkcije je niz koji sadrži vrijednosti funkcije gubitka za sve čestice u roju, koje nastojimo minimizirati.

```
def f(x):
    n_particles = x.shape[0]
    values = [forward_prop(x[i]) for i in range(n_particles)]
    return np.array(values)
```

Proces formiranja objektivne funkcije je kompletiran, tako da preostaje optimizacija, koju provodimo ponovo u skladu sa primjerima iznad. Parametar `dimensions` je određen prema dimenzijama mreže, kako je objašnjeno u jednom od prethodnih pasusa.

```
%%time
options = {'c1': 0.5, 'c2': 0.3, 'w':0.9}

dimensions = (n_inputs * n_hidden) + (n_hidden * n_classes) + n_hidden + n_classes
optimizer = ps.single.GlobalBestPSO(n_particles=100, dimensions=dimensions, options=
    options)

cost, pos = optimizer.optimize(f, iters=100)
```

Postavlja se pitanje kako provjeriti rad mreže. Kod standardno treniranih i strukturiranih mreža, mogli smo koristiti metode modela, bez obraćanja previše pažnje na sam proces. Međutim, testiranje mreže nije ništa drugo nego propuštanje novih, dosad neviđenih podataka, kroz mrežu. Funkcija `probs_function` za nas simulira prolazak nekog skupa podataka kroz mrežu odgovarajuće strukture. Bitno je samo da joj kao parametre proslijedimo najbolji rezultat procesa optimizacije (tj. najbolji set težina i otklona), te skup testnih podataka. Na taj način formiramo jednostavnu predikcijsku funkciju datu ispod.

```
def predict(pos):
    probs = probs_function(pos, X_test)
    return probs
```

Rezultate testiranja je onda moguće provjeriti u jednoj liniji koda:

```
(np.round(predict(pos)).astype('int') == y_test).mean()
```

Vjerovatnoće dobijene predikcijom se zaokružuju na najbližu cjelobrojnu vrijednost, te se na taj način odlučuje kojoj klasi pripada testni podatak prema modelu. Taj se rezultat poredi sa stvarnim labelama, te se kao rezultat poziva `mean` metode nad uslovom dobija broj u rasponu [0, 1] koji govori za koji je dio skupa uslov zadovoljen.